

FreshRSS avec Docker : se remettre proprement à la veille

David LEGRAND

13 mai 2026

“Quand on bosse dans le numérique, on sait tous qu’il faut faire de la veille. Sur le papier, c’est évident. En pratique, c’est vite le bazar.

On a des articles à droite, des blogs à gauche, des posts LinkedIn, des newsletters, des sites institutionnels, des chaînes YouTube, des dépôts GitHub, des docs officielles... Résultat : on passe parfois plus de temps à chercher l’info qu’à réellement la lire.

C’est là que FreshRSS devient intéressant.

L’idée est simple : au lieu d’aller visiter 25 sites un par un, on centralise leurs flux RSS ou Atom dans un seul outil. FreshRSS va récupérer les nouveaux articles automatiquement, les ranger par catégorie, et nous permettre de les consulter tranquillement depuis une interface web.

En gros, c’est un lecteur RSS que l’on peut héberger soi-même.

Pourquoi FreshRSS ?

FreshRSS a plusieurs avantages.

Déjà, c’est libre. On garde la main sur ses données et on ne dépend pas d’un service externe qui peut changer ses règles, devenir payant ou disparaître.

Ensuite, c’est sobre. Ça ne cherche pas à remplacer un réseau social, ni à nous pousser du contenu en permanence. On choisit nos sources, on lit ce qu’on veut, quand on veut.

Et surtout, c’est très pratique pour organiser une veille professionnelle. Par exemple, on peut créer des catégories comme :

- IA
- Python
- Développement web
- Linux
- RGPD / CNIL / AI Act
- Pédagogie
- Cybersécurité

Puis ajouter les flux correspondant aux sites que l’on veut suivre.

Pour moi, l’intérêt est surtout là : construire petit à petit un environnement de veille fiable, sans dépendre uniquement des algorithmes ou des réseaux sociaux.

Pourquoi l’installer avec Docker ?

FreshRSS peut s’installer de manière classique sur un serveur web, mais Docker simplifie franchement les choses.

On évite de gérer à la main PHP, le serveur web, les extensions, les droits sur les dossiers, etc. L’application tourne dans un conteneur, avec tout ce dont elle a besoin.

Autre avantage : la configuration tient dans un fichier `compose.yml`. Pour lancer, arrêter ou mettre à jour FreshRSS, quelques commandes suffisent.

Pour un usage personnel, on peut très bien utiliser SQLite. Pas besoin de sortir directement MariaDB ou PostgreSQL si on veut juste centraliser sa veille.

Préparer le dossier du projet

Sur le serveur, on peut commencer par créer un dossier dédié :

```
mkdir -p ~/docker/freshrss
cd ~/docker/freshrss
```

Ensuite, on crée un fichier `compose.yml`.

Exemple simple de `compose.yml`

Voici une version simple, utilisable rapidement :

```
services:
  freshrss:
    image: freshrss/freshrss:latest
    container_name: freshrss
    restart: unless-stopped

    environment:
      TZ: Europe/Paris
      CRON_MIN: '3,33'

    volumes:
      - freshrss_data:/var/www/FreshRSS/data
      - freshrss_extensions:/var/www/FreshRSS/extensions

    ports:
      - "8080:80"

volumes:
  freshrss_data:
  freshrss_extensions:
```

Ici, FreshRSS sera disponible sur le port 8080 de la machine.

On lance le conteneur avec :

```
docker compose up -d
```

Puis on peut accéder à FreshRSS avec :

```
http://adresse-du-serveur:8080
```

À partir de là, l'installation se termine dans le navigateur.

Version avec Caddy et un nom de domaine

Si on veut faire quelque chose de plus propre, on peut mettre FreshRSS derrière un reverse proxy comme Caddy.

Par exemple, avec le domaine :

```
rss.david-legrand.fr
```

et un réseau Docker externe nommé `cloudnet`, on peut utiliser cette version :

```
services:
  freshrss:
```

```
image: freshrss/freshrss:latest
container_name: freshrss
restart: unless-stopped

environment:
  TZ: Europe/Paris
  CRON_MIN: '3,33'
  BASE_URL: https://rss.david-legrand.fr

volumes:
  - freshrss_data:/var/www/FreshRSS/data
  - freshrss_extensions:/var/www/FreshRSS/extensions

networks:
  - cloudnet

volumes:
  freshrss_data:
  freshrss_extensions:

networks:
  cloudnet:
    external: true
```

Dans ce cas, on ne publie pas de port avec `ports`. FreshRSS reste accessible uniquement depuis le réseau Docker interne, et c'est Caddy qui fait le lien avec l'extérieur.

Côté Caddy, un bloc simple peut suffire :

```
rss.david-legrand.fr {
  reverse_proxy freshrss:80
}
```

Caddy s'occupe ensuite du HTTPS automatiquement, à condition que le domaine pointe bien vers le serveur.

Première configuration

Quand on arrive sur FreshRSS pour la première fois, l'interface demande quelques informations :

Langue
Base de données
Compte administrateur
Mot de passe
URL de l'instance

Pour une installation personnelle, SQLite fait très bien le travail. C'est simple, léger, et ça évite d'ajouter un conteneur de base de données.

Une fois l'installation terminée, on peut commencer à créer ses catégories et à ajouter ses flux.

Organiser sa veille sans se noyer

Le piège avec un agrégateur RSS, c'est de vouloir tout ajouter d'un coup.

Mauvaise idée.

On se retrouve vite avec 500 articles non lus, et ça devient aussi décourageant qu'une boîte mail pleine.

Le mieux est de commencer petit :

Quelques sources fiables sur l'IA
Quelques sources Python

Quelques sources institutionnelles pour la réglementation
Quelques blogs techniques

Puis on ajuste au fur et à mesure.

Si un flux ne sert jamais, on le supprime. Si une source publie trop souvent du contenu peu utile, on la met de côté. Le but n'est pas de tout lire, mais de mieux repérer ce qui vaut vraiment le coup.

Mettre à jour FreshRSS

La mise à jour est classique avec Docker Compose :

```
docker compose pull
docker compose up -d
```

Les données restent dans les volumes Docker, donc elles ne disparaissent pas quand le conteneur est recréé.

Cela dit, avant une grosse mise à jour, une sauvegarde reste une bonne habitude.

Sauvegarder les données

Pour sauvegarder le volume des données :

```
docker run --rm \
  -v freshrss_data:/data \
  -v "$PWD":/backup \
  alpine \
  tar czf /backup/freshrss_data_backup.tar.gz -C /data .
```

Et pour sauvegarder les extensions :

```
docker run --rm \
  -v freshrss_extensions:/extensions \
  -v "$PWD":/backup \
  alpine \
  tar czf /backup/freshrss_extensions_backup.tar.gz -C /extensions .
```

Ce n'est pas la partie la plus fun, mais le jour où quelque chose casse, on est bien content de les avoir.

Et les extensions ?

FreshRSS permet aussi d'ajouter des extensions.

Dans notre configuration, elles sont stockées dans ce volume :

```
- freshrss_extensions:/var/www/FreshRSS/extensions
```

Cela permet de les conserver même après une mise à jour du conteneur.

On peut donc enrichir FreshRSS progressivement, sans forcément tout installer dès le départ.

Conclusion

FreshRSS, ce n'est pas l'outil le plus spectaculaire du monde. Et justement, c'est ce qui le rend intéressant.

Il fait une chose, il la fait bien : centraliser des flux d'information pour nous aider à faire une veille plus propre.

Avec Docker, l'installation est assez simple, la maintenance reste raisonnable, et on peut obtenir rapidement une instance accessible depuis un vrai nom de domaine.

Pour moi, c'est typiquement le genre de petit outil que l'on installe un jour "pour tester", puis qu'on finit par utiliser régulièrement parce qu'il rend vraiment service.